
30-Day AWS Portfolio Project Plan

From Zero to Job-Ready: A Day-by-Day Cloud Engineering Roadmap

Prepared for: **Quell** | May 13, 2026 | Gallatin, TN

AWS **Terraform** **Python 3.12** **IaC** **DevOps**



Mission Statement:

This plan transforms 30 days of hands-on AWS work into a recruiter-ready cloud engineering portfolio — progressing across four phases from foundational networking and compute, through scalable serverless architectures, into full production-grade infrastructure automated entirely with Terraform and CloudFormation. By Day 30, you will have 6+ deployable, documented projects ready to show in any AWS job interview.



Table of Contents

- 1. Overview & Goals
- 2. Phase 1 — Foundations (Days 1-7)
 - Day 1: AWS Account Setup & IAM Best Practices

Day 2: VPC Networking Deep Dive
Day 3: EC2 Web Server Deployment
Day 4: S3 Static Website + Bucket Policies
Day 5: IAM Roles, Policies & EC2 Instance Profiles
Day 6: CloudWatch Monitoring & Billing Alerts
Day 7: Week 1 Review & GitHub Commit
3. Phase 2 — Intermediate Projects (Days 8–14)
Days 8–14: ALB, ASG, CloudFront, Lambda, API Gateway, DynamoDB
4. Phase 3 — Advanced Production Architecture (Days 15–21)
Days 15–21: RDS, Secrets Manager, 3-Tier VPC, X-Ray, SQS/SNS, WAF
5. Phase 4 — IaC, Portfolio Polish & Final Build (Days 22–30)
Days 22–30: Terraform, SAM, GitHub Structure, Diagrams, Final Deploy
6. GitHub README Template
7. AWS Services Quick Reference
8. Daily Time Budget
9. Cost Management Guide
10. Terraform Module Reference
11. Interview Prep Cheat Sheet
12. 30-Day Progress Tracker

Section 1: Overview & Goals

Portfolio Objective

Build **6+ production-quality AWS projects** across foundational, intermediate, and advanced tiers — culminating in a polished GitHub portfolio with resume-ready bullet points, professional architecture diagrams, and fully automated Infrastructure as Code. This is not a tutorial-following exercise — every project is designed to demonstrate real engineering decision-making to hiring managers and technical interviewers.

Tools & Technologies Covered

Category	Tools
Cloud Console & CLI	AWS Management Console, AWS CLI v2
Infrastructure as Code	Terraform v1.9+, AWS CloudFormation, AWS SAM CLI
Version Control	Git, GitHub (portfolio repo: <code>aws-portfolio</code>)
Diagramming	Draw.io, Lucidchart (AWS Architecture Icon set)
IDE & Extensions	VS Code + AWS Toolkit + HashiCorp Terraform Extension
API Testing	Postman, curl

AWS Services Covered

VPC, **EC2**, **ALB**, **Auto Scaling Groups**, **S3**, **CloudFront**, **Lambda**, **API Gateway**, **DynamoDB**, **RDS MySQL**, **IAM**, **CloudWatch**, **SNS**, **SQS**, **Route 53**, **ACM (SSL)**, **AWS Secrets Manager**, **GuardDuty**, **WAF**, **CloudTrail**, **AWS Config**

Weekly Milestone Summary

Week	Theme	Key Deliverables
Week 1 Days 1-7	 Foundations & VPC Networking	VPC with public/private subnets, EC2 web server, S3 static site, IAM least-privilege setup, CloudWatch alarms
Week 2 Days 8-14	 Intermediate: Scalable & Serverless	EC2 + ALB + Auto Scaling Group, S3 + CloudFront CDN, Lambda + API Gateway REST API, DynamoDB CRUD
Week 3 Days 15-21	 Advanced: Production Architecture	RDS in private subnets, multi-tier VPC, Secrets Manager, CloudWatch + X-Ray observability, SNS/SQS pipelines, WAF
Week 4 Days 22-30	 IaC, Portfolio Polish & Final Build	Terraform/CloudFormation full-stack deploy, GitHub structure, professional

Week	Theme	Key Deliverables
		architecture diagrams, README polish, interview prep

Section 2: Phase 1 — Foundations

Days 1-7 | Core AWS services, networking basics, and your first deployable project

Day 1 — AWS Account Setup & IAM Best Practices

- Create AWS Free Tier account at aws.amazon.com
- Enable **MFA on root account** immediately — use an authenticator app
- Create an **IAM admin user** (never use root for day-to-day work)
- Create IAM groups: `Admins`, `Developers`, `ReadOnly`
- Attach least-privilege policies to each group
- Install **AWS CLI v2**, configure with `aws configure` using your admin user keys
- Install **Terraform v1.9+** and verify: `terraform version`
- Install **VS Code** with AWS Toolkit and HashiCorp Terraform extensions
- GitHub: Create portfolio repo `aws-portfolio` with initial folder structure
- Verify CLI works: `aws sts get-caller-identity`

 Deliverable

Screenshot of IAM dashboard with MFA enabled, CLI output of `aws sts get-caller-identity`, first GitHub commit with repo scaffold.

Pro Tip

Enable AWS Cost Anomaly Detection on Day 1 — it will alert you if spending unexpectedly spikes. Never use your root account again after today; treat it like a break-glass key.

Day 2 — VPC Networking Deep Dive

- Create a custom VPC: CIDR `10.0.0.0/16`
- Create 2 public subnets: `10.0.1.0/24` (us-east-1a), `10.0.2.0/24` (us-east-1b)
- Create 2 private subnets: `10.0.3.0/24` (us-east-1a), `10.0.4.0/24` (us-east-1b)
- Create Internet Gateway (IGW) and attach to VPC
- Create NAT Gateway in public subnet (attach Elastic IP)
- Configure route tables: public → `0.0.0.0/0` → IGW, private → `0.0.0.0/0` → NAT
- Associate subnets with correct route tables
- Configure Network ACLs and Security Groups with least-privilege rules

Deliverable

VPC architecture diagram in Draw.io (export as PNG), screenshot of VPC dashboard showing all components green.

Pro Tip

Always build VPCs across

2+ Availability Zones

— even in dev. Hiring managers will ask why, and the answer demonstrates production thinking. This VPC becomes the backbone of every project this month.

Day 3 — EC2 Web Server Deployment

- Launch **EC2 t2.micro** in public subnet using Amazon Linux 2023 AMI
- Write User Data script to auto-install Apache/Nginx and serve a custom HTML page on launch
- Attach Security Group: port 80 (HTTP from 0.0.0.0/0), 443 (HTTPS), 22 (SSH from your IP only)
- Assign **Elastic IP** to the instance
- Create EC2 key pair, download and store safely (`chmod 400 mykey.pem`)

- SSH into instance: `ssh -i mykey.pem ec2-user@<elastic-ip>`
- Verify web server is running: `curl http://localhost`

Deliverable

Public Elastic IP serving your custom HTML page in browser — take a screenshot showing the URL and page content.

Security Note

SSH port 22 must be restricted to
your IP only
in the Security Group inbound rules — never
0.0.0.0/0
for SSH. This is a common interview question and a real-world
misconfiguration that gets companies breached.

Day 4 — S3 Static Website + Bucket Policies

- Create S3 bucket for static hosting (name it after your domain if available)
- Enable **static website hosting** on the bucket
- Upload an HTML/CSS portfolio index page as the entry point
- Configure bucket policy for public read access (or use CloudFront OAC —

previewed on Day 10)

- Enable **S3 Versioning** — protect against accidental overwrites
- Enable **server-side encryption** (SSE-S3, AES-256)
- Enable access logging to a separate `access-logs-bucket`

Deliverable

S3 website endpoint URL serving your portfolio page. Screenshot of bucket settings showing versioning, encryption, and logging all enabled.

Pro Tip

On Day 10 you will lock this bucket down and serve it exclusively through CloudFront — the public read policy here is temporary. Document this decision in your README as a learning evolution.

Day 5 — IAM Roles, Policies & EC2 Instance Profiles

- Create a custom IAM policy: allow EC2 to `s3:GetObject`, `s3:PutObject`, `s3:ListBucket` on your specific S3 bucket only
- Create an **IAM Role for EC2** with that policy attached and EC2 as the trusted entity
- Attach the role to your EC2 instance via **instance profile**
- Test from EC2: `aws s3 ls s3://your-bucket-name` — must work with zero

access keys configured

- Create a separate IAM policy for least-privilege DynamoDB access (for Day 11)
- Document all IAM decisions and policy JSON in your GitHub README

Deliverable

EC2 successfully accessing S3 via IAM role — no hardcoded credentials anywhere. Screenshot of successful

aws s3 ls

from inside the EC2 instance.

Pro Tip

The

Instance Metadata Service (IMDS)

is how EC2 fetches temporary credentials from the attached IAM role.

Know how to explain this in an interview: the role → instance profile → IMDS → temporary STS token chain.

Day 6 — CloudWatch Monitoring & Billing Alerts

- Create CloudWatch **billing alarms** at \$5 and \$20 thresholds (in us-east-1)

- Create EC2 CPU utilization alarm: CPU > 80% for 5 consecutive minutes → SNS notification
- Create an **SNS topic** (aws-alerts), subscribe your email, confirm subscription
- Enable **detailed monitoring** on EC2 instance (1-minute resolution)
- Create a CloudWatch **Dashboard**: widgets for EC2 CPU, NetworkIn/Out, StatusCheckFailed
- Set up **VPC Flow Logs** to CloudWatch Logs log group /vpc/flowlogs

Deliverable

CloudWatch dashboard screenshot with at least 4 widgets. Confirmation email from SNS showing your subscription. Billing alarm visible in CloudWatch Alarms console.

Cost Note

Detailed EC2 monitoring costs ~\$3.50/month per instance. Enable it on your demo instance, but disable it on instances you leave idle. VPC Flow Logs charges for data ingestion — clean up log groups weekly.

Day 7 — Week 1 Review & GitHub Commit

- Review all Week 1 resources in the AWS Console — verify everything is as

expected

- Write `README.md` for each project folder: description, services used, architecture, what you learned
- Draw final Week 1 architecture diagram: VPC, subnets, EC2, IGW, NAT Gateway, S3
- Tag **all AWS resources**: `Environment: Dev, Project: Week1, Owner: Quell`
- Cost review: open AWS Cost Explorer, understand what drove charges this week
- Push all Week 1 code, diagrams, and READMEs to GitHub

Deliverable

GitHub repo with all Week 1 project folders populated with READMEs, architecture diagrams (PNG), and any configuration files. All resources tagged.

Section 3: Phase 2 — Intermediate Projects

Days 8-14 | Scalable compute, content delivery, serverless APIs, and NoSQL

Day 8 — Application Load Balancer (ALB) Setup

- Create a **second EC2 instance** (t2.micro) in the second public subnet

(different AZ)

- Install web server on both instances; serve different "Server: Instance-1" and "Server: Instance-2" pages to visualize load balancing
- Create a **Target Group** (protocol: HTTP, port: 80) and register both EC2 instances
- Create an **ALB** spanning both public subnets with an internet-facing scheme
- Create ALB Listener: HTTP:80 → forward to Target Group
- Test ALB DNS name in browser — refresh repeatedly to verify round-robin routing between instances
- Configure ALB access logs to S3 (enable after creating the bucket with correct permissions)

Deliverable

ALB DNS name serving traffic alternately from both instances. Screenshot showing Instance-1 and Instance-2 responses. ALB access logs appearing in S3.

Pro Tip

The ALB operates at

Layer 7 (HTTP/HTTPS)

— it can route based on path, host header, or query string. Know the difference between ALB (L7), NLB (L4), and CLB (legacy) for interviews.

Day 9 — Auto Scaling Group (ASG)

- Create a **Launch Template** from your EC2 AMI (include User Data script)
- Create an **Auto Scaling Group**: min=2, desired=2, max=4, spanning both public subnets
- Attach ASG to the ALB Target Group
- Configure **Scaling Policies**: Scale Out: CPU > 70% for 2 minutes → add 1 instance Scale In: CPU < 30% for 10 minutes → remove 1 instance
- Test scale-out: SSH into an instance and run `stress --cpu 4 --timeout 300`
- Configure ASG health check type to **ELB** (not EC2)
- Configure **Instance Refresh** for zero-downtime deployments (min healthy %: 90)

Deliverable

Screenshot of ASG Activity History showing a successful scale-out event.
CloudWatch graph showing CPU spike triggering the alarm.

Day 10 — S3 + CloudFront CDN

- Create a private S3 bucket (Block All Public Access enabled)
- Upload your static website or SPA build files
- Create a **CloudFront distribution**: Origin: S3 bucket with **Origin Access Control (OAC)** Redirect HTTP to HTTPS Enable Compress Objects

automatically Price Class: North America and Europe (cost saving)

- Request **ACM SSL certificate** in us-east-1 (required for CloudFront)
- Configure custom error response: 404 → /index.html, HTTP 200 (SPA routing)
- Update S3 bucket policy to allow only your specific CloudFront distribution via OAC

Deliverable

CloudFront URL serving your site over HTTPS. Verify the S3 bucket direct URL is blocked (403 Forbidden). Screenshot of CloudFront distributions console showing your distribution Deployed.

Pro Tip

OAC replaces the legacy OAI

— always use OAC in new distributions. Be ready to explain how

aws:SourceArn

condition in the S3 bucket policy restricts access to exactly one CloudFront distribution.

Day 11 — AWS Lambda Functions

Write three Lambda functions in **Python 3.12**:

- `get-items` — scans and returns all items from a DynamoDB table
- `post-item` — accepts a JSON body, writes a new item to DynamoDB
- `delete-item` — deletes an item by ID from DynamoDB
- Configure Lambda **environment variables**: `TABLE_NAME`, `AWS_REGION`
- Assign IAM execution role with DynamoDB read/write permissions (least privilege)
- Test each function with test events directly in the Lambda console
- Enable **Lambda CloudWatch logging** — verify logs appear in `/aws/lambda/` log group
- Configure memory: 256MB, timeout: 30 seconds

Deliverable

All three Lambda functions tested with  green status. CloudWatch Logs showing successful invocations. Screenshot of Lambda console with all three functions.

Day 12 — API Gateway + Lambda Integration

- Create a **REST API** in API Gateway
- Create resources and methods with **Lambda Proxy Integration**:
`GET /items` → `get-items` Lambda
`POST /items` → `post-item` Lambda
`DELETE /items/{id}` → `delete-item` Lambda

- Enable **CORS** on all resources (Access-Control-Allow-Origin: *)
- Deploy API to a dev stage
- Enable API Gateway **access logging** to CloudWatch
- Test with Postman: verify GET returns items, POST creates an item, DELETE removes it

Deliverable

Postman collection with all three endpoints tested. Working REST API URL. Screenshot showing successful 200 responses for GET, POST (201), and DELETE operations.

Day 13 — DynamoDB Design & Best Practices

- Create DynamoDB table: Items with partition key `id` (String)
- Add a **Global Secondary Index (GSI)**: `status-index` on attribute `status`
- Enable **Point-in-Time Recovery (PITR)** — production best practice
- Configure **DynamoDB Auto Scaling**: min 1 RCU/WCU, max 10 RCU/WCU
- Write 10 sample items using the Lambda POST endpoint (test your full API)
- Query items by status using the GSI via the Lambda get-items function
- Enable **DynamoDB Streams** — foundation for event-driven patterns (Day 19)

Deliverable

DynamoDB table with 10+ items, GSI defined, PITR enabled, Streams active. Screenshot of DynamoDB console showing table metrics.

Pro Tip

Know the difference between
DynamoDB on-demand mode
(pay-per-request, no capacity planning) vs.
provisioned mode
(fixed RCU/WCU with auto-scaling). You're using provisioned here to
demonstrate auto-scaling skills.

Day 14 — Week 2 Review, Architecture Diagram & GitHub Push

- Draw comprehensive Week 2 architecture diagram: ALB → ASG → EC2 (both AZs), CloudFront → S3, API Gateway → Lambda → DynamoDB
- Write detailed READMEs for each Week 2 project folder
- Export Postman collection (JSON) and commit to GitHub alongside Lambda source code
- Create **GitHub Issues** for improvements you want to revisit (WAF, custom domain, monitoring)

- Review costs for Week 2 in Cost Explorer

Deliverable

GitHub repo updated with all Week 2 projects, READMEs, Postman collection, and architecture diagrams. GitHub Issues created for future improvements.

Section 4: Phase 3 — Advanced Production Architecture

Days 15–21 | Production-grade databases, security, observability, and event-driven design

Day 15 — RDS MySQL in Private Subnets

- Create a **DB Subnet Group** using both private subnets (multi-AZ ready)
- Create **RDS MySQL 8.0** (db.t3.micro, Free Tier eligible): Place in private subnets — **no public accessibility** Multi-AZ: disabled (cost), but document the toggle and why you'd enable it in production Enable automated backups: 7-day retention window Enable **encryption at rest** (AWS-managed key)
- Create RDS Security Group: allow port 3306 **only from the EC2 Security Group ID**
- Connect from EC2 bastion: `mysql -h <rds-endpoint> -u admin -p`
- Create a sample database and table; insert test rows

Deliverable

RDS instance running in private subnet (no public endpoint). Successful MySQL connection from EC2 bastion. Screenshot of RDS console showing instance Available.

Cost Warning

RDS db.t3.micro runs ~\$0.017/hr (~\$12/month if left on).

Stop the RDS instance

when you're not actively working — you can stop it for up to 7 days at a time from the console.

Day 16 — Secrets Manager & Parameter Store

- Store RDS credentials in **AWS Secrets Manager** (JSON secret, auto-rotation ready)
- Store application config (DB name, table names) in **SSM Parameter Store** as SecureString
- Update EC2 IAM role: add `secretsmanager:GetSecretValue` and `ssm:GetParameter` permissions
- Write Python script on EC2 that retrieves RDS password from Secrets

Manager and connects to RDS — **zero hardcoded credentials**

- Store Lambda environment variable references as SSM parameters, retrieve at runtime

Deliverable

Python script on EC2 successfully connecting to RDS using credentials fetched dynamically from Secrets Manager. No credentials in code, environment variables, or config files. Commit the script to GitHub.

Pro Tip

Use

Secrets Manager for credentials that rotate

(DB passwords, API keys) and

SSM Parameter Store for app config

that doesn't rotate (DB names, bucket names, region). This distinction matters in interviews.

Day 17 — Multi-Tier VPC Architecture (Full Production Layout)

Redesign your VPC to reflect a true **3-tier production architecture**:

Tier

Subnet Type

Resources

Public Tier

Public subnets

ALB, NAT Gateway, Bastion Host

App Tier

Private subnets

EC2 (ASG), Lambda (VPC config)

Data Tier

Private/isolated subnets

RDS, DynamoDB via VPC Endpoint

- Create **VPC Gateway Endpoint for S3** (free — traffic stays within AWS backbone)
- Document Interface Endpoint for Secrets Manager (~\$7/mo — deploy, demo, then destroy)
- Configure Lambda in VPC (private subnets) with execution role accessing DynamoDB via VPC endpoint
- Implement **Security Group chaining**: ALB SG → EC2 SG → RDS SG (least privilege, no IP ranges in SG rules)

Deliverable

Full 3-tier architecture diagram documenting all layers, all security groups, and all traffic flows. Each Security Group rule documented with a reason in

your README.

Day 18 — CloudWatch Observability & X-Ray Tracing

- Create a **CloudWatch Log Group** for each service with appropriate retention (30 days)
- Create metric filters to detect HTTP 5xx errors in ALB access logs
- Create a **CloudWatch Composite Alarm**: (HTTP 500 rate > 1% AND latency > 2s) → SNS alert
- Enable **AWS X-Ray** active tracing on all Lambda functions
- Enable X-Ray tracing on API Gateway stage settings
- View the **X-Ray Service Map** — visualize the call graph: API Gateway → Lambda → DynamoDB
- Create a unified **CloudWatch Dashboard**: ALB latency, Lambda duration/errors/throttles, DynamoDB consumed RCU/WCU, RDS connections

Deliverable

CloudWatch Dashboard screenshot with 6+ widgets. X-Ray Service Map screenshot showing your full request chain with latency breakdown per segment.

Day 19 — SQS & SNS Event-Driven Pattern

- Create an **SQS Standard Queue** and a **Dead Letter Queue (DLQ)**
- Configure DLQ: `maxReceiveCount=3` (message moved to DLQ after 3 failed processing attempts)
- Create a Lambda function triggered by SQS (Event Source Mapping)
- Create an SNS topic that fans out to the SQS queue and to your email
- Build the full event pipeline: API Gateway POST → publishes message to SNS SNS → fans out to SQS queue + email SQS → triggers Lambda → Lambda writes to DynamoDB
- Send a test message through the pipeline and verify it appears in DynamoDB

Deliverable

End-to-end event-driven pipeline working with a single API call triggering the full chain. Architecture diagram of the pipeline. Screenshot of DynamoDB showing the item created via the async pipeline.

Pro Tip

This pattern (fan-out) is one of the most commonly asked AWS architecture questions. Know that

SNS → SQS

decouples producers from consumers, enables retry logic via DLQ, and allows multiple independent consumers to process the same event.

Day 20 — Security Hardening & WAF Basics

- Enable **AWS Config** with managed rules: `ec2-instance-no-public-ip`, `s3-bucket-public-read-prohibited`, `rds-instance-public-access-check`
- Enable **Amazon GuardDuty** (30-day free trial — enable today)
- Apply **S3 Block Public Access** at the account level
- Create **IAM Access Analyzer** for the account
- Create and attach **AWS WAF Web ACL** to your ALB: AWS Managed Rule: `AWSMangedRulesCommonRuleSet` Rate-based rule: max 1,000 requests per IP per 5 minutes
- Verify **CloudTrail** is recording all API calls to an S3 bucket (enable if not done on Day 1)

Deliverable

GuardDuty active (screenshot). AWS Config rules all showing Compliant (green). WAF Web ACL attached to ALB. IAM Access Analyzer showing no external access findings.

Day 21 — Week 3 Review & Full Architecture Diagram

- Draw the **complete production architecture diagram** covering all three tiers, all services deployed in Weeks 1-3
- Document each major design decision with an explicit reason (why private subnets for RDS? why Secrets Manager over env vars? why OAC over OAI?)
- Write Week 3 READMEs for each project folder
- Review and clean up all unused resources — check Cost Explorer for surprises
- Push all Week 3 work to GitHub, update the main portfolio README

Deliverable

GitHub updated with Week 3 content. Full production architecture diagram (your most impressive visual asset) saved as both PNG and SVG.

Architecture decision log committed to

`docs/architecture-decisions.md`

.

Section 5: Phase 4 — IaC, Portfolio Polish & Final Build

Days 22-30 | Automate everything, publish your portfolio, prepare for interviews

Day 22 — Terraform Foundations

Establish your Terraform project structure:

```
terraform/ ├── main.tf ├── variables.tf ├── outputs.tf ├── providers.tf ├──  
modules/ |   ├── vpc/ |   ├── ec2/ |   ├── alb/ |   ├── rds/ ├── environments/  
          |── dev/   |── prod/
```

- Write `providers.tf`: AWS provider `~> 5.0`, required Terraform version `~> 1.9`
- Configure **remote state**: S3 backend with versioning enabled + DynamoDB table for state locking
- Write the `vpc` module: VPC, public/private subnets, IGW, NAT Gateway, route tables
- Run `terraform init`, `terraform plan`, `terraform apply`
- Verify VPC in console matches your Terraform configuration exactly

Deliverable

VPC deployed via Terraform with state stored in S3.

`terraform plan`

shows "No changes" after apply. GitHub commit with all Terraform files.

Pro Tip

Never store

`terraform.tfstate`

in Git

— add it to

.gitignore

. Always use remote state in production. The S3 + DynamoDB backend pattern is the industry standard and a common interview topic.

Day 23 — Terraform: EC2, ALB & ASG Modules

- Write `ec2` module: Launch Template, Security Group, key pair
- Write `alb` module: ALB, Target Group, Listeners, Security Group
- Write ASG resource using the Launch Template and ALB Target Group attachment
- Parameterize everything via `variables.tf`: instance type, AMI ID, min/max capacity, environment name
- Add outputs: ALB DNS name, EC2 instance IDs, VPC ID, subnet IDs
- Run `terraform apply` and verify the ALB DNS is serving traffic

Deliverable

ALB + ASG deployed entirely via Terraform — zero manual console clicks.
ALB DNS name in Terraform outputs. GitHub commit.

Day 24 — Terraform: Lambda, API Gateway & DynamoDB

- Write Terraform for Lambda using data `"archive_file"` to package Python source as zip
- Write API Gateway REST API resources in Terraform (methods, integrations, deployments, stages)
- Write DynamoDB table resource with GSI, auto-scaling policies, PITR enabled
- Wire Lambda execution role with inline policies for DynamoDB access
- Full terraform `apply` — complete serverless stack deployed via IaC, zero manual steps

Deliverable

Full serverless stack (Lambda + API Gateway + DynamoDB) deployed via Terraform. API endpoint in Terraform outputs, tested with Postman.

Day 25 — CloudFormation Alternative (AWS SAM for Serverless)

- Install **AWS SAM CLI**: `pip install aws-sam-cli`
- Write `template.yaml` for SAM: `AWS::Serverless::Function` for each Lambda handler `AWS::Serverless::Api` for API Gateway `AWS::DynamoDB::Table`
- Build and deploy: `sam build` then `sam deploy --guided`
- Deploy a SAM-based Hello World API to a staging environment stack
- Write a comparison document: **SAM vs Terraform tradeoffs**
(CloudFormation-native vs multi-cloud, YAML vs HCL, serverless-first vs

general-purpose)

Deliverable

SAM-deployed serverless API with a named CloudFormation stack.

Comparison document

docs/sam-vs-terraform.md

committed to GitHub.

Day 26 — GitHub Repository Structure & Documentation

Final GitHub repository structure:

```
aws-portfolio/ |—— README.md                    ← Main portfolio README |——
project-1-vpc-networking/ | |—— README.md | |—— architecture/ | | |——
vpc-diagram.png | |—— terraform/ |—— project-2-ec2-alb-asg/ | |——
README.md | |—— architecture/ | |—— terraform/ |—— project-3-s3-cloudfront/
| |—— README.md | |—— terraform/ |—— project-4-lambda-api-dynamodb/ |
|—— README.md | |—— src/ | | |—— get_items.py | | |—— post_item.py
| | |—— delete_item.py | |—— terraform/ | |—— sam-template.yaml |——
project-5-rds-private/ | |—— README.md | |—— terraform/ |—— project-6-
production-architecture/ | |—— README.md | |—— architecture/ | | |——
production-architecture.png | |—— terraform/ |—— docs/ |—— architecture-
decisions.md |—— cost-breakdown.md
```

Deliverable

Polished, professional GitHub repository ready to share in job applications.

Pin the

aws-portfolio

repo on your GitHub profile.

Day 27 — Architecture Diagrams with Draw.io / Lucidchart

Create professional diagrams for all 6 projects using the official AWS icon set:

Color

Represents

Services

 Orange

Compute

EC2, Lambda, ASG, ECS

 Blue

Networking

VPC, ALB, CloudFront, Route 53, IGW, NAT

 Green

Storage

S3, RDS, EBS

● Purple

Serverless / Messaging

Lambda, API Gateway, DynamoDB, SNS, SQS

- Use official AWS Architecture Icons (free from aws.amazon.com/architecture/icons/)
- Include AZ boundaries, VPC boundary, subnet labels (public/private)
- Show data flow arrows with labels: HTTP, HTTPS, SQL, AMQP, gRPC
- Export each diagram as both **PNG** (embed in README) and **SVG** (for editing)
- Embed diagrams directly in each project's README using standard Markdown image syntax

Deliverable

6 professional architecture diagrams, one per project folder, embedded in their respective READMEs. Diagrams follow AWS icon standards and color coding.

Day 28 — Final Production Build: Full-Stack AWS Application

Deploy the complete production architecture from `environments/prod/` using Terraform:

- Custom VPC with 3-tier architecture
- ALB + Auto Scaling Group (app tier, private subnets)

- CloudFront + S3 with Origin Access Control (CDN layer)
- API Gateway + Lambda + DynamoDB (serverless API, VPC-integrated Lambda)
- RDS MySQL in private data-tier subnets
- Secrets Manager for all credentials
- CloudWatch dashboard + composite alarms
- SNS + SQS event-driven async pipeline
- Run `terraform apply` from `environments/prod/` — watch the full stack provision
- Verify all services are working end-to-end with a full smoke test
- Take screenshots of every service in the console — these are your portfolio screenshots

Deliverable

Fully deployed production architecture, all services verified working. Folder of console screenshots for portfolio use. Terraform output file committed to GitHub.

Day 29 — Resume Bullet Points & Portfolio Write-Up

Write strong, quantified resume bullets for each project. Examples to customize:

- "Designed and deployed a **highly available 3-tier AWS architecture** using VPC with public/private subnets, ALB, Auto Scaling Groups, and RDS MySQL — achieving zero single points of failure"

- "Built a **serverless REST API** using Lambda, API Gateway, and DynamoDB, handling full CRUD operations with IAM least-privilege execution roles and CloudWatch observability"
- "Automated **100% of infrastructure provisioning** using Terraform modules with remote S3 state and DynamoDB locking — eliminating all manual console configuration"
- "Implemented S3 + CloudFront CDN with Origin Access Control, ACM SSL certificates, and custom error pages — delivering static content with **sub-100ms global latency**"
- "Configured AWS WAF, GuardDuty, IAM Access Analyzer, and AWS Config rules — establishing a **defense-in-depth security posture** across the entire environment"
- "Built an **event-driven async pipeline** using SNS fan-out to SQS + Lambda with Dead Letter Queue — demonstrating production-grade fault tolerance patterns"
- Update your LinkedIn About section with AWS project highlights
- Write a 1-paragraph description per project for cover letter use

Deliverable

5+ resume bullets per project ready to paste into your resume. LinkedIn profile updated. GitHub profile bio updated with AWS portfolio link.

Day 30 — Portfolio Review, Cleanup & Interview Prep

- Final walkthrough: verify every diagram matches the actual deployed

architecture

- Run `terraform destroy` on costly resources to stop charges

Keep Running (Low Cost)

Destroy (Hourly Charges)

S3 static site, CloudFront, Lambda (pay-per-use), DynamoDB (free tier forever)

RDS instance, NAT Gateway, EC2 instances, ALB

- Targeted destroy: `terraform destroy -target=aws_nat_gateway.main`
- Pin `aws-portfolio` repo on your GitHub profile
- Review the interview Q&A cheat sheet in Section 11

Deliverable

Interview-ready portfolio with pinned GitHub repos, updated resume, and CloudWatch billing alert confirmed at \$5 threshold. You are job-ready.

Section 6: GitHub README Template

Use the following structure for your main `aws-portfolio/README.md`. This is the first thing a recruiter or hiring manager sees when they click your GitHub link — make it count.

Main `aws-portfolio` README Structure

1. **Header:** Your name + "AWS Cloud Engineer Portfolio" + location (Gallatin, TN)
2. **Badges:** AWS, Terraform, Python, GitHub Actions, License

3. **"About This Portfolio"** — 2 sentences about your learning journey and what you built

4. **Projects Table:**

#	Project	AWS Services	Difficulty
1	VPC Networking	VPC, EC2, IGW, NAT Gateway, Security Groups	★★ Intermediate
2	EC2 + ALB + Auto Scaling	EC2, ALB, ASG, Launch Template, CloudWatch	★★★ Advanced
3	S3 + CloudFront CDN	S3, CloudFront, ACM, OAC, Route 53	★★ Intermediate
4	Serverless REST API	Lambda, API Gateway, DynamoDB, IAM	★★★ Advanced
5	RDS in Private Subnets	RDS MySQL, Secrets Manager, VPC, Bastion	★★★★★ Expert
6	Production Architecture	All services above + Terraform + WAF + X-Ray + SNS/SQS	★★★★★ Production

5. **Skills & Tools:** AWS, Terraform, Python 3.12, Git, Draw.io, Postman, VS Code

6. **Architecture Diagrams:** Embed PNGs directly from the `architecture/` subfolders

7. **Contact:** LinkedIn URL, professional email

Section 7: 🏡 AWS Services Quick Reference

Service	Description	Days Used
VPC	Isolated virtual network —	2, 17, 22

Service	Description	Days Used
	foundational container for all AWS resources	
EC2	Virtual servers — scalable compute instances in any size	3, 8, 9
ALB	Application Load Balancer — Layer 7 HTTP/HTTPS traffic routing with path-based rules	8, 9, 23
Auto Scaling Group	Automatically adjust EC2 fleet size based on CloudWatch metrics and schedules	9, 23
S3	Object storage for static files, backups, logs, and Terraform remote state	4, 10, 22
CloudFront	Global CDN with 400+ edge locations — caches content for sub-100ms latency worldwide	10, 24
Lambda	Serverless compute — run code without provisioning servers, billed per 100ms	11, 19, 24
API Gateway	Managed REST/HTTP API front door — handles auth, throttling, CORS, staging	12, 24
DynamoDB	Serverless NoSQL database — single-digit millisecond latency, auto-scaling capacity	13, 24
RDS MySQL	Managed relational database in private subnet — automated backups, patching, failover	15
Secrets Manager	Secure credential storage with automatic rotation via Lambda — no hardcoded passwords	16
SSM Parameter Store	App config and lightweight secrets — SecureString type uses KMS encryption	16
IAM	Identity and access management — roles, policies, groups, and least-privilege enforcement	1, 5

Service	Description	Days Used
CloudWatch	Metrics, logs, alarms, dashboards — unified observability for all AWS services	6, 18
SNS	Pub/sub fan-out service — one message to multiple subscribers simultaneously	6, 19
SQS	Managed message queue with DLQ support — decouples producers from consumers	19
WAF	Web Application Firewall — rate limiting, managed rule sets, geo-blocking	20
GuardDuty	ML-powered threat detection — detects compromised credentials, crypto mining, recon	20
ACM	Free SSL/TLS certificates for AWS resources — auto-renews, zero management overhead	10
Route 53	DNS service with health checks, routing policies (latency, weighted, failover)	10
CloudTrail	API audit log for every AWS call — who did what, when, from where	20
Terraform	HashiCorp IaC — multi-cloud, HCL modules, remote state, plan/apply workflow	22-25
AWS SAM	Serverless Application Model — CloudFormation extension for Lambda-first IaC	25
X-Ray	Distributed tracing — visualize request chains across Lambda, API Gateway, DynamoDB	18

Section 8: Daily Time Budget

Recommended daily schedule for 2–4 hours of focused work. Consistency beats marathon sessions — 2.5 hours every day outperforms 10 hours on weekends.

Time Block	Activity	Duration
Block 1	Review yesterday's work, read today's plan, set up AWS console tabs	30 minutes
Block 2	Hands-on AWS Console / CLI / Terraform work — the core build session	90 minutes
Block 3	Write README documentation, commit code and config to GitHub	45 minutes
Block 4	Update or create architecture diagram in Draw.io / Lucidchart	30 minutes
Block 5	 Daily cost check in AWS Cost Explorer — catch runaway charges early	15 minutes

Time Management Tip

Set a

daily calendar block

as non-negotiable. If you can only do 90 minutes on a given day, prioritize Block 2 (the hands-on build) and at minimum push a single commit to GitHub. Streaks matter — recruiters look at GitHub contribution graphs.

Section 9: 💰 Cost Management Guide

Expected total cost over 30 days: \$0-\$15 if you follow the destroy discipline below. The biggest risk is forgetting to stop RDS instances and NAT Gateways.

Key Cost Drivers

Service	Cost	Action
NAT Gateway	~\$0.045/hr + \$0.045/GB data processed (~\$33/mo if left on)	⚠ Destroy when not actively working
RDS db.t3.micro	~\$0.017/hr (~\$12/mo if always on)	⚠ Stop instance when idle — max 7 days stopped
EC2 t2.micro	750 hrs/mo free (first 12 months of account)	✅ Free Tier — watch if you have multiple instances
ALB	~\$0.008/hr + LCU costs (~\$6/mo baseline)	⚠ Destroy when done with a phase
CloudFront	1TB/mo transfer free (first 12 months)	✅ Free Tier — safe to leave running
Lambda	1M free requests/mo, 400K GB-seconds/mo (permanent free tier)	✅ Leave running — essentially free at this scale
DynamoDB	25 GB storage, 25 WCU, 25 RCU free forever	✅ Permanent free tier — no action needed
S3	5 GB storage free (first 12 months)	✅ Clean up large files after projects
Secrets Manager	\$0.40/secret/month — minimal	✅ Delete secrets after projects

Money-Saving Commands

```
# Destroy only the NAT Gateway (save ~$33/mo) terraform destroy -  
target=aws_nat_gateway.main # Destroy only the RDS instance terraform  
destroy -target=aws_db_instance.main # Destroy entire dev environment cd  
environments/dev && terraform destroy # Check current month's spend via CLI  
aws ce get-cost-and-usage \ --time-period Start=2026-05-01,End=2026-05-31 \  
--granularity MONTHLY \ --metrics BlendedCost
```

Pro Cost Strategy

Enable

AWS Cost Anomaly Detection

(free service) on Day 1. It uses ML to detect unusual spend patterns and will email you before a surprise bill. Set a budget alert at \$5 in AWS Budgets as a hard backstop.

Section 10: ⚙️ Terraform Module Reference

VPC Module — `modules/vpc/main.tf`

```
resource "aws_vpc" "main" {  cidr_block      = var.vpc_cidr
enable_dns_hostnames = true  enable_dns_support = true    tags =
merge(var.tags, {          Name = "${var.project_name}-vpc"    }) } resource
"aws_subnet" "public" {  count      = length(var.public_subnet_cidrs)
vpc_id      = aws_vpc.main.id  cidr_block      =
var.public_subnet_cidrs[count.index]  availability_zone =
var.availability_zones[count.index]  map_public_ip_on_launch = true    tags
= merge(var.tags, {          Name = "${var.project_name}-public-${count.index +
1}"          Tier = "Public"    }) } resource "aws_internet_gateway" "main"
{  vpc_id = aws_vpc.main.id  tags = merge(var.tags, { Name = "${
var.project_name}-igw" }) } resource "aws_nat_gateway" "main"
{  allocation_id = aws_eip.nat.id  subnet_id      = aws_subnet.public[0].id
tags      = merge(var.tags, { Name = "${var.project_name}-nat" })
depends_on = [aws_internet_gateway.main] } resource "aws_eip" "nat"
{  domain = "vpc"  tags = merge(var.tags, { Name = "${var.project_name}-
nat-eip" }) }
```

Lambda Function — Terraform

```
data "archive_file" "lambda_zip" {  type      = "zip"  source_dir = "${
path.module}/src"  output_path = "${path.module}/lambda_function.zip" }
resource "aws_lambda_function" "api_handler" {  filename      =
data.archive_file.lambda_zip.output_path  function_name      = "${
var.project_name}-api-handler"  role      =
aws_iam_role.lambda_exec.arn  handler      = "get_items.lambda_handler"
runtime      = "python3.12"  source_code_hash =
data.archive_file.lambda_zip.output_base64sha256  memory_size      = 256
timeout      = 30  environment {  variables = {          TABLE_NAME =
```



```
aws_dynamodb_table.items.name      REGION      = var.aws_region      } }
tracing_config { mode = "Active"    # Enables X-Ray tracing    } tags =
var.tags }
```

Python Lambda Handler — get_items.py

```
import json import boto3 import os from boto3.dynamodb.conditions import Key
dynamodb = boto3.resource('dynamodb') table =
dynamodb.Table(os.environ['TABLE_NAME']) def lambda_handler(event, context):
try: response = table.scan() items = response.get('Items',
[]) return { 'statusCode': 200, 'headers': {
'Content-Type': 'application/json', 'Access-Control-Allow-
Origin': '*' }, 'body': json.dumps(items) }
except Exception as e: print(f"Error: {str(e)}") return {
'statusCode': 500, 'body': json.dumps({'error':
str(e)}) }
```

Remote State Backend — providers.tf

```
terraform { required_version = "~> 1.9" required_providers { aws = {
source = "hashicorp/aws" version = "~> 5.0" } } backend "s3"
{ bucket = "quell-terraform-state" key = "aws-
portfolio/terraform.tfstate" region = "us-east-1" encrypt
= true dynamodb_table = "terraform-state-lock" } } provider "aws"
{ region = var.aws_region default_tags { tags = { Project
= "aws-portfolio" Owner = "Quell" Environment =
var.environment ManagedBy = "Terraform" } } }
```

Section 11: Interview Prep Cheat Sheet

Master these 15 questions and you will handle the AWS technical portion of cloud engineering interviews with confidence. Each answer references work you actually built during this 30-day plan.

Q1. "What is a VPC and why do you use public and private subnets?"

A VPC is an isolated virtual network within AWS where you control IP addressing, routing, and access controls. Public subnets have a route to the Internet Gateway — used for resources that need to accept inbound internet traffic (ALBs, bastion hosts). Private subnets route outbound-only through a NAT Gateway — ideal for databases and app servers that should never be directly reachable from the internet. In my portfolio, I built a 3-tier VPC with this exact separation: ALB in public, EC2 in private app tier, RDS in private data tier.

Q2. "Explain how an Application Load Balancer works with Auto Scaling."

The ALB distributes incoming HTTP/HTTPS traffic across registered EC2 targets using listener rules and target groups. The Auto Scaling Group manages the EC2 fleet — it automatically registers new instances with the ALB Target Group when scaling out, and deregisters them (with connection draining) when scaling in. In my project, I configured ELB health checks on the ASG so that unhealthy instances are replaced automatically, enabling zero-downtime Instance Refresh for deployments.

Q3. "How does Lambda pricing work and when would you NOT use Lambda?"

Lambda charges per request (first 1M free forever) and per GB-second of duration. It's extremely cost-effective for event-driven, short-duration workloads. I'd avoid Lambda for: tasks exceeding the 15-minute timeout, workloads requiring persistent TCP connections, and latency-sensitive paths where cold starts are unacceptable (mitigated with Provisioned Concurrency or container images). In my portfolio, I used Lambda for DynamoDB CRUD — perfectly suited to short, stateless operations.

Q4. "What is the difference between SQS and SNS?"

SNS is pub/sub — one message fans out to all subscribers simultaneously (email, SQS, Lambda, HTTP). SQS is a queue — messages are stored and consumed by one consumer at a time, with retry logic, visibility timeout, and Dead Letter Queue support for failed messages. The common production pattern — which I built on Day 19 — is SNS fanout to multiple SQS queues, giving each downstream consumer its own retry and DLQ without affecting others.

Q5. "How do you manage secrets in AWS?"

AWS Secrets Manager for database credentials and API keys — it supports automatic rotation via Lambda and charges \$0.40/secret/month. SSM Parameter Store SecureString for app configuration (DB names, endpoints) — free for standard parameters, encrypted with KMS. Critically, I never hardcode credentials anywhere: EC2 instances use IAM instance profiles, Lambda uses IAM execution roles, and both fetch secrets at runtime via the AWS SDK. No credentials ever touch source code or environment variables directly.

Q6. "What is the difference between CloudFormation and Terraform?"

CloudFormation is AWS-native — no state file to manage, AWS handles state internally, supports drift detection, and is deeply integrated with all AWS services. Terraform uses HCL, maintains a state file (remote state in S3 + DynamoDB locking), has a rich open-source module registry, and supports multi-cloud. For AWS-only environments, CloudFormation is sufficient and has tighter service parity. I used both: Terraform for the full infrastructure stack and AWS SAM (CloudFormation extension) for the serverless API — comparing tradeoffs in my docs.

Q7. "What is a NAT Gateway and why is it in the public subnet?"

A NAT Gateway allows private subnet resources to initiate outbound internet connections (downloading OS updates, calling external APIs) without exposing them to inbound traffic from the internet. It must live in a public subnet with an Elastic IP because it itself needs a route to the Internet Gateway to forward traffic. It translates the private IP to its public Elastic IP for the outbound request, then routes the response back to the originating private resource.

Q8. "How do you make an S3 bucket secure while using it with CloudFront?"

Use Origin Access Control (OAC) — the modern replacement for OAI. Keep the S3 bucket fully private with Block Public Access enabled. Create a CloudFront distribution with

OAC, which generates a signed request identity for your specific distribution. Update the S3 bucket policy to allow `s3:GetObject` only when `aws:SourceArn` matches your exact CloudFront distribution ARN. All traffic flows through CloudFront over HTTPS — direct S3 access returns 403.

Q9. "What is DynamoDB auto-scaling and when do you need it?"

DynamoDB auto-scaling adjusts read/write capacity units based on a target utilization percentage (typically 70%). When consumed capacity exceeds the target, auto-scaling increases RCU/WCU; when utilization drops, it scales in. Use it for provisioned mode tables with unpredictable traffic patterns. For truly spiky or unknown workloads, DynamoDB On-Demand mode is simpler — pay per request, no capacity planning — though it costs more at consistent high volume. I configured provisioned mode with auto-scaling to demonstrate both concepts.

Q10. "How does RDS Multi-AZ work and how is it different from Read Replicas?"

RDS Multi-AZ maintains a synchronous standby replica in a different AZ. All writes are committed to both the primary and standby before returning success. On failure, RDS automatically promotes the standby with ~60-120 second failover — this is for high availability, not performance. Read Replicas are asynchronous copies used to offload read-heavy workloads to improve performance — they can be in different regions. Read Replicas can be promoted manually but don't provide automatic failover.

Q11. "What is an IAM instance profile?"

An instance profile is a container that associates an IAM role with an EC2 instance. When EC2 starts, it fetches temporary credentials from the Instance Metadata Service at `169.254.169.254/latest/meta-data/iam/security-credentials/`. These STS credentials rotate automatically every hour. The AWS SDK uses them transparently — no access

key ID or secret key is needed on the server. This is the security-correct way to give EC2 access to S3, DynamoDB, Secrets Manager, and other services.

Q12. "Explain the difference between Security Groups and NACLs."

Security Groups are stateful (return traffic is automatically allowed), operate at the resource/ENI level, and are allow-only (you can't write explicit deny rules). NACLs are stateless (you must explicitly allow both inbound and outbound), operate at the subnet level, and support both allow and deny rules with numbered priority ordering. Security Groups are your primary defense layer; NACLs add subnet-level protection. In my VPC, I chain Security Groups: ALB SG → EC2 SG → RDS SG using security group IDs as sources — never IP ranges.

Q13. "What is CloudWatch X-Ray and how do you use it?"

AWS X-Ray provides distributed tracing across services. Enable active tracing on Lambda via the execution role (xray:PutTraceSegments, xray:PutTelemetryRecords) and toggle X-Ray in Lambda configuration. Enable it on API Gateway in stage settings. X-Ray generates a service map showing the full request chain (API Gateway → Lambda → DynamoDB), with per-segment latency, error rates, and throttle rates. It's essential for debugging performance bottlenecks and unexpected errors in microservice architectures.

Q14. "How does Terraform remote state work?"

Remote state stores your `terraform.tfstate` file in S3 (versioned, encrypted at rest) instead of locally. A DynamoDB table provides state locking — when anyone runs `terraform plan` or `apply`, Terraform writes a lock record to DynamoDB. Anyone else attempting to run Terraform simultaneously gets an error, preventing state corruption. Configure this in the `terraform` block with `backend "s3"`, specifying the bucket, key path, region, and DynamoDB table name. Never commit `.tfstate` to Git.

Q15. "What would you change in your architecture to make it fully production-ready?"

I'd add: RDS Multi-AZ enabled, AWS Shield Standard (free, already on), WAF on ALB (done), Route 53 health checks with failover routing, ACM wildcard certificate for the domain, VPC Flow Logs to S3 with Athena for querying, CloudTrail to S3 with CloudWatch Events for compliance, GuardDuty (done), AWS Config conformance packs (done), and a full CI/CD pipeline using GitHub Actions or CodePipeline — zero manual deployments. I'd also enforce IMDSv2 on all EC2 instances and enable S3 Object Lock for audit trail buckets.

Section 12: 30-Day Progress Tracker

Print this section or keep it open as your daily dashboard. Check off each day as you complete it.

Day	Focus Area	Key Task	Status
1	IAM & Setup	Create IAM user + groups, enable MFA, configure CLI, install Terraform, create GitHub repo	☐
2	VPC Networking	Custom VPC, public/private subnets, IGW, NAT Gateway, route tables, security groups	☐
3	EC2 Deployment	Launch EC2 in public subnet, User Data script, Elastic	☐

Day	Focus Area	Key Task	Status
		IP, SSH access, web server verified	
4	S3 Static Site	S3 bucket, static hosting, bucket policy, versioning, encryption, access logging	☐
5	IAM Roles	EC2 instance profile, least-privilege S3 policy, test from EC2 without access keys	☐
6	CloudWatch	Billing alarms (\$5/\$20), EC2 CPU alarm, SNS topic, CloudWatch Dashboard, VPC Flow Logs	☐
7	Week 1 Review	Tag all resources, write READMEs, draw architecture diagram, GitHub push, cost review	☐
8	ALB Setup	Second EC2 instance, Target Group, ALB in both AZs, listener, test round-robin, access logs to S3	☐
9	Auto Scaling Group	Launch Template, ASG (min=2, max=4), scaling policies, stress test, ELB health check, Instance Refresh	☐
10	CloudFront CDN	Private S3 bucket, CloudFront distribution with OAC, ACM cert, HTTPS redirect, custom 404	☐
11	Lambda Functions	3 Python 3.12 Lambdas (get/post/delete), IAM execution role, test events, CloudWatch logging	☐
12	API Gateway	REST API, Lambda	☐

Day	Focus Area	Key Task	Status
		proxy integration, CORS, deploy to dev stage, test all methods in Postman	
13	DynamoDB	Table + GSI, PITR enabled, auto-scaling, 10 sample items, DynamoDB Streams enabled	☐
14	Week 2 Review	Week 2 architecture diagram, READMEs, Postman collection export, GitHub push, GitHub Issues	☐
15	RDS MySQL	DB Subnet Group, RDS in private subnets, encrypted, bastion connect via MySQL client	☐
16	Secrets Manager	Store RDS creds in Secrets Manager, app config in SSM, Python script retrieving secrets dynamically	☐
17	3-Tier VPC	Full production VPC layout, VPC endpoints (S3 gateway), SG chaining (ALB→EC2→RDS)	☐
18	Observability	Log groups, metric filters, composite alarm, X-Ray on Lambda + API Gateway, CloudWatch Dashboard	☐
19	SQS + SNS Pipeline	SQS + DLQ, SNS topic, Lambda SQS trigger, full pipeline: API → SNS → SQS → Lambda → DynamoDB	☐
20	Security Hardening	GuardDuty, AWS Config rules, S3 Block Public Access, IAM Access Analyzer, WAF on	☐

Day	Focus Area	Key Task	Status
		ALB, CloudTrail	
21	Week 3 Review	Complete production architecture diagram, architecture decision log, GitHub push, cleanup	☐
22	Terraform VPC	Terraform project structure, providers.tf, S3 remote state + DynamoDB lock, VPC module, terraform apply	☐
23	Terraform EC2/ALB	ec2 module, alb module, ASG resource, parameterized variables, outputs, terraform apply	☐
24	Terraform Serverless	Lambda (archive_file), API Gateway, DynamoDB table via Terraform, full serverless stack apply	☐
25	AWS SAM	SAM template.yaml, sam build, sam deploy --guided, staging stack, SAM vs Terraform comparison doc	☐
26	GitHub Polish	Final repo structure, all 6 project READMEs, docs/ folder, main portfolio README	☐
27	Architecture Diagrams	6 professional diagrams with AWS icons, color coding, AZ boundaries, data flow arrows, embedded in READMEs	☐
28	Final Production Build	Full prod deploy via terraform apply (environments/prod/	☐

Day	Focus Area	Key Task	Status
		), smoke test all services, console screenshots	
29	Resume & LinkedIn	5+ bullet points per project, LinkedIn About updated, cover letter descriptions written	☐
30	Cleanup & Interview Prep	Destroy costly resources, pin GitHub repos, review Q&A cheat sheet, verify billing alarm active	☐

Final Words — You've Got This, Quell

By the end of Day 30, you will have built infrastructure that many mid-level engineers haven't touched. The combination of **6 documented projects**, **professional architecture diagrams**, **Terraform IaC**, and **a polished GitHub portfolio** puts you ahead of the overwhelming majority of cloud engineering candidates.

The secret is simple: **ship something every single day** and commit it to GitHub. Recruiters see contribution graphs. Interviewers see GitHub repos. Hiring managers see architecture diagrams. All three will see someone who builds things — and that is exactly who gets hired.

Start Day 1 today. The cloud waits for no one. 🏔

30-Day AWS Portfolio Project Plan | Quell | Gallatin, TN | May 2026 | All AWS services subject to pricing changes — verify current rates at aws.amazon.com/pricing